

Consumer 장애 Runbook 유형별 대응 절차와 모니터링

장애 유형 4가지 · 모니터링 지표 설계 · 자동화 확장

강의 30min + 실습 25min

ConsumerMonitorService

장애 4유형

NotifierAdapter

Runbook

Consumer 장애 4가지 유형 — 증상만 같고 조치가 다르다

FOUR FAILURE TYPES — SAME SYMPTOM, DIFFERENT ROOT CAUSE, DIFFERENT FIX

1 프로세스 크래시

PM2 status → stopped/errored

메시지 처리 전면 중단

조치: pm2 restart

2 Consumer Lag 증가

프로세스 살아있음

pending_messages 지속 증가

조치: 병목 탐색 → 확장

3 역등성 실패 반복

DLQ 재큐잉 후 또 DLQ 진입

duplicate key 에러 반복

조치: 원장 확인 → drop or 코드 수정

4 Redis 연결 끊김

ECONNREFUSED / ETIMEDOUT

모든 처리 중단

조치: Redis 확인 → 인프라팀

핵심 — 유형을 잘못 판단하면 조치가 역효과를 낸다. Lag에 pm2 restart = PEL 재처리 부하 추가로 오히려 악화.

유형 1 — 프로세스 크래시

CONSUMER PROCESS CRASH — PM2 RESTART + PEL AUTO-RECLAIM

증상 및 원인

증상 — PM2 status: stopped/errored, 처리 전면 중단

원인

- unhandledRejection (처리 안 된 예외)
- OOM (메모리 초과)
- 외부 종료 시그널

```
# 즉각 조치
pm2 restart issuer-consumer

# 크래시 원인 확인
pm2 logs issuer-consumer --lines 100

# 재시작 횟수 확인
pm2 show issuer-consumer

# 복구 확인
curl /admin/queue/stats
# → consumerCount >= 1 확인
```

PEL 재수신이 왜 자동인가

- 1 XREADGROUP으로 가져간 메시지
→ PEL에 등록됨
- 2 XACK 전에 Consumer 크래시
→ PEL에 잔류
- 3 재시작 후 XAUTOCLAIM으로
PEL 메시지 자동 재수신
- 4 멍등성 덕분에 중복 없이 재처리

주의 — 같은 크래시 3회 이상 반복 = 코드 버그. 재시작 루프 멈추고 배포팀 에스컬.

유형 2 — Consumer Lag 증가

LAG INCREASE — PROCESS IS ALIVE BUT SLOW, DO NOT RESTART

Lag 임계값

- 정상 — `pending_messages < 50`
- 주의 — `50 ≤ pending < 200` → Slack 알림
- 경고 — `200 ≤ pending < 500` → 담당자 확인
- 긴급 — `pending ≥ 500` → 즉시 대응

원인 탐색 순서

```
# 1. VASP 응답 시간 확인
curl /admin/vasp/health
# p99 > 5s → VASP 지연이 원인

# 2. DB 슬로우 쿼리 확인
SELECT pid, duration, query
FROM pg_stat_activity
WHERE state = 'active'
ORDER BY duration DESC LIMIT 10;

# 3-a. VASP 지연 → Consumer 확장
pm2 start issuer-consumer \
  --name issuer-consumer-2

# 3-b. DB 병목 → DBA 에스컬레이션
```

Lag에 pm2 restart는 효과 없음. 오히려 PEL 재처리 부하 추가로 악화.

유형 3 — 멍등성 실패 반복 (DLQ 재진입)

IDEMPOTENCY FAILURE — REQUEUE AFTER DLQ LEADS BACK TO DLQ

증상 및 원인

증상

- requeue해도 또 DLQ 진입
- duplicate key violates unique constraint

원인

- Consumer 코드 버그 (상태 전이 오류)
- 멍등성 키 설계 오류
- 외부 시스템이 다른 ID로 두 번 발행

조치 순서

1 DLQ 메시지 원본 확인

GET /admin/dlq/pending

2 processed_events 에서 txHash 조회

3a 이미 처리됨 → drop

원장 반영 완료, 재처리 불필요

3b 미처리 + 코드 버그 → 수정 배포 후 requeue

외부 중복 발행 → 발신 시스템 팀 통보

원인 파악 없이 requeue 금지 — 무한 루프. 코드 버그가 원인이면 배포 없이 requeue는 의미 없다.

유형 4 — Redis 연결 끊김

REDIS CONNECTION LOSS — SELF-HEALING FORBIDDEN, ESCALATE TO INFRA

증상 및 원인

증상

- Error: connect ECONNREFUSED
- ETIMEDOUT 반복
- 모든 처리 중단

원인

- Redis 서버 다운/재시작
- 네트워크 파티션
- maxmemory 초과 → 연결 거부

확인 순서

```
# 1. Redis 응답 확인
redis-cli ping

# PONG → Redis 정상 → Consumer 재시작
# 연결 실패 → Redis 문제 → 아래 진행

# 2. 메모리 사용량 확인
redis-cli info memory | grep used_memory_human

# 90% 이상 → 인프라팀 즉시 호출

# 3. 클라이언트 연결 수 확인
redis-cli info clients | grep connected_clients
```

⚠ Redis 문제 자체 해결 시도 금지

Redis 다운 → 즉시 인프라팀 에스컬레이션
연락처: infra@kyobo.com

Redis 정상이면 — Consumer만 재시작. Redis 문제이면 — 인프라팀. 판단 기준: redis-cli ping 결과.

모니터링 지표 6가지 — 유형별 조기 감지

SIX METRICS — EACH TARGETS A SPECIFIC FAILURE TYPE

지표	감지 대상 장애	임계값	우선순위
consumer_process_up	프로세스 크래시 (유형 1)	= 0 즉시	P1
redis_connected	Redis 끊김 (유형 4)	= false 즉시	P1
pending_messages	Lag 증가 (유형 2)	>200 경고 / >500 긴급	P2 / P1
dlq_count	역등성 실패 (유형 3)	>0 알림	P2
processing_rate	Lag 증가 보조 지표	<10/s 주의	P2
pel_size	미ACK 메시지 누적	>100 경고	P2

P1 알림 — consumer_process_up=0, redis_connected=false, pending≥500. 세 경우 모두 즉시 대응. 야간 호출 트리거.

ConsumerMonitorService — 지표 수집 구현

GETSTAT() + CHECKALERTS() — METRICS COLLECTION AND THRESHOLD ALERTING

```
export interface ConsumerStats {
  streamLength: number; // 전체 스트림 길이
  pendingMessages: number; // 미처리 (Lag)
  pelSize: number; // PEL: 미ACK
  dlqCount: number; // DLQ 적재 수
  consumerCount: number; // 활성 Consumer 수
  redisConnected: boolean;
}
```

```
async getStats(): Promise<ConsumerStats> {
  const [streamLen, groupInfo, dlqItems] =
    await Promise.all([
      this.redis.xlen(this.streamKey),
      this.redis.xInfoGroups(this.streamKey),
      this.dlqHandler.listPending(),
    ]);
```

```
  const group = groupInfo.find(
    g => g.name === this.groupName);
  const redisConnected =
    await this.redis.ping()
      .then(() => true)
      .catch(() => false);
  return {
    streamLength: streamLen,
    pendingMessages: group?.pending ?? 0,
    pelSize: group?.pending ?? 0,
    dlqCount: dlqItems.length,
    consumerCount: group?.consumers ?? 0,
    redisConnected,
  };
}
```

```
async checkAlerts(
  stats: ConsumerStats
): Promise<string[]> {
  const alerts: string[] = [];

  if (!stats.redisConnected)
    alerts.push('P1: Redis 연결 끊김');
  if (stats.consumerCount === 0)
    alerts.push('P1: Consumer 없음(크래시)');
  if (stats.dlqCount > 0)
    alerts.push(`P2: DLQ ${stats.dlqCount}건`);
  if (stats.pendingMessages >= 500)
    alerts.push(`P1: Lag 긴급`);
  else if (stats.pendingMessages >= 200)
    alerts.push(`P2: Lag 경고`);

  return alerts;
}
```

라우터 연결

GET /admin/queue/stats

→ stats + alerts 함께 반환

→ 운영자가 한 번에 상황 파악

장애 대응 5대 원칙 — Runbook의 전제

FIVE PRINCIPLES — VIOLATE THESE AND THE RUNBOOK FAILS

1

/admin/queue/stats 먼저

유형 확인 전에 조치 금지. 증상이 같아도 원인이 다르면 조치가 역효과.

2

프로세스 재시작은 유형 1에만

Lag(유형 2)에 pm2 restart = PEL 재처리 부하 추가 → 오히려 악화.

3

DLQ requeue 전 원장 확인

이미 processed_events에 있으면 drop. 확인 없이 requeue = 무한 루프.

4

Redis 다운은 자체 해결 시도 금지

즉시 인프라팀 에스컬레이션. Redis 직접 조작은 데이터 손실 위험.

5

모든 운영 행위는 감사 로그

requeue/drop API가 자동 처리. 야간 장애에서도 추적 가능해야 한다.

Consumer 장애 대응 워크시트

WORKSHEET — FILLING ORDER = DIAGNOSIS ORDER

1단계: 초기 상태 확인 (무조건 여기서 시작)

GET /admin/queue/stats 결과:

consumerCount: ____ (0이면 → 유형 1)

pendingMessages: ____ (200+ → 유형 2)

dlqCount: ____ (0+ → 유형 3 병행)

redisConnected: ☐ true ☐ false (false → 유형 4)

2단계: 유형 판정

☐ 유형 1 — 프로세스 크래시

☐ 유형 2 — Lag 증가

☐ 유형 3 — 멍등성 실패 반복

☐ 유형 4 — Redis 연결 끊김

☐ 복합 — 유형 ____ + 유형 ____

3단계: 원인 파악 + 4단계: 조치

☐ pm2 restart issuer-consumer ____:____

☐ Consumer 인스턴스 추가 ____:____

☐ DLQ requeue ____:____

☐ DLQ drop ____:____

☐ 인프라팀 에스컬레이션 ____:____

5단계: 사후 확인 (10분 후)

consumerCount: ____ (1 이상 정상)

pendingMessages: ____ (감소 추세 정상)

redisConnected: ☐ true

☐ 정상 복구 ☒ ☐ 미해소 → 에스컬레이션

NotifierAdapter — 알림 채널 교체 패턴

PLUG-IN NOTIFIER — SWAP CHANNEL WITH ONE ENV VAR

인터페이스 (채널 불문)

```
export interface NotifierAdapter {
  // 즉시 알림 - 장애 감지 시
  sendAlert(payload: AlertPayload): Promise<void>;

  // 인시던트 생성 알림
  sendIncidentAlert(incident: Incident): Promise<void>;

  // 인시던트 종료 알림
  sendIncidentResolved(
    incidentId: string, note: string
  ): Promise<void>;

  // 주간 리포트
  sendWeeklyReport(report: WeeklyReport): Promise<void>;
}

// 채널 선택 - 환경변수 1개
function buildNotifier(): NotifierAdapter {
  switch (process.env.NOTIFIER_TYPE) {
    case 'slack': return new SlackNotifierAdapter(...);
    case 'teams': return new TeamsNotifierAdapter(...);
    default:     return new ConsoleNotifierAdapter();
  }
}
```

구현체 교체 — 코드 변경 없음

Slack

SlackNotifierAdapter
Webhook URL

Teams

TeamsNotifierAdapter
Webhook URL

Email

EmailNotifierAdapter
SMTP Config

DIP 적용 — IncidentService는 NotifierAdapter 인터페이스에만 의존.
교보 측 협업 도구 확정 시 NOTIFIER_TYPE 환경변수만 변경. 코드 변경 0.

WeeklyReportService — 주간 자동 집계

WEEKLY AUTO-REPORT — PATTERN DETECTION FROM INCIDENT DB

자동 집계 항목

유형별 인시던트 건수 + 평균 해소 시간

DLQ requeue / drop 건수

동일 root_cause 3회+ 반복 패턴 자동 감지

```
// cron 등록 - 매주 월요일 09:00 KST
cron.schedule('0 9 * * 1', () =>
  weeklyReport.sendWeeklyReport(), {
    timezone: 'Asia/Seoul',
  });
```

WeeklyReport 구조

```
interface WeeklyReport {
  weekLabel:      string;    // "2026-05-04 ~ 2026-05-10"
  totalIncidents: number;
  avgResolutionMinutes: number;
  dlqTotal:       number;
  dlqRequeued:    number;
  dlqDropped:     number;
  typeBreakdown:  string;    // "DLQ:3 / LAG:2 / CRASH:1"
  repeatPatterns: string;    // 동일 원인 3회+ 항목
}
```

반복 패턴 — HAVING COUNT(*) >= 3 쿼리로 자동 감지. 반복 패턴 = 코드/인프라 개선 신호.

아날로그 워크시트 → 디지털 자동화 대응

PAPER TO DIGITAL — WHAT'S AUTOMATED vs WHAT NEEDS HUMAN JUDGMENT

아날로그 (워크시트)	디지털 (API / 서비스)	자동화 여부
초기 상태 확인 빈칸	POST /admin/incidents 응답의 snapshot	☑ 자동
txHash 원장 중복 확인	snapshot.dlqItems[].alreadyInLedger	☑ 자동
조치 기록 (4단계)	POST /admin/incidents/:id/actions	운영자 입력
사후 확인 (5단계)	GET /admin/queue/stats 재조회	운영자 확인
주간 리뷰 시트 집계	WeeklyReportService.generate()	☑ 자동
반복 패턴 분석	HAVING COUNT(*) >= 3 쿼리	☑ 자동
알림 채널 선택	NOTIFIER_TYPE 환경변수 교체	환경변수 1개

자동화 철학 — 시스템이 수집할 수 있는 것은 시스템이. 운영자는 판단이 필요한 부분(원인 분류, requeue/drop 결정)에 집중.

S52 완료 기준

ACCEPTANCE CRITERIA — MONITORING + RUNBOOK DOCUMENTATION

☐ GET /admin/queue/stats — stats + alerts 필드 포함 응답 확인

☐ Consumer 강제 중단 → **consumerCount: 0** + **P1 알림** 확인

☐ DLQ 메시지 적재 → **dlqCount > 0** + **P2 알림** 확인

☐ **pendingMessages ≥ 200** → P2: Consumer Lag 경고 알림 확인

☐ Runbook 문서 docs/runbooks/consumer-incident-runbook.md — 4가지 유형 모두 포함

에스컬레이션 기준 — Lag 500+ + 30분 지속 → 팀장 호출 / Redis 다운 → 즉시 인프라팀 + 팀장 / DLQ 20건+ → 팀장 호출

에스컬레이션 기준표

ESCALATION MATRIX — WHEN TO CALL WHO

상황	조치	연락처
Consumer 재시작으로 5분 내 해소	자체 해결 후 리포트	자체 해결
Lag 500+ & 30분 지속	서비스 관리자 호출	팀장
DLQ 20건 이상	서비스 관리자 호출	팀장
Redis 다운	즉시 인프라팀 + 팀장	infra팀 즉시
동일 크래시 3회 이상 반복	코드 버그 → 배포팀	배포팀
2회 DLQ 재진입	원인 재분석 → 배포팀	배포팀

P1 — 즉시 대응 (야간 호출 가능)

consumer_process_up = 0
redis_connected = false
pending_messages ≥ 500

P2 — 업무 시간 대응

dlq_count > 0
pending_messages 200~499
processing_rate < 10/s

P3 — 모니터링 / 리포트

인시던트 해소 알림
주간 리포트
정상 복구 확인

S52 핵심 정리

KEY TAKEAWAYS — CONSUMER RUNBOOK AND MONITORING DESIGN

장애 4유형 — 유형부터 판단

- 1 크래시 → pm2 restart
- 2 Lag → 병목 탐색 → 확장
- 3 멍등성 실패 → 원장 확인 후 drop/requeue
- 4 Redis 끊김 → 인프라팀 즉시

모니터링 핵심 지표

consumer_count=0 → P1 크래시
redis_connected=false → P1 Redis
pending≥500 → P1 Lag
dlq_count>0 → P2 멍등성

NotifierAdapter 패턴

NOTIFIER_TYPE 환경변수 1개로
Slack / Teams / Email 교체
코드 변경 0

S52의 핵심은 코드가 아니라 **판단 기준**이다.

/admin/queue/stats 먼저 → 유형 판정 → 유형별 조치 — 이 순서만 지키면 야간 장애에서도 실수하지 않는다.